Raport z realizacji zadania rekrutacyjnego

PWR Racing Team

Contents

1 Wstęp **3**

1.1 Użyte narzędzia . . . 3

2 Zadanie 1 **4**

2.1 Algorytm Zachłanny (Greedy Clustering) . . . 4

2.1.1 Zasada działania algorytmu . . . 4

2.1.2 Złożoność obliczeniowa i optymalizacje . . . 4

2.1.3 Implementacja . . . 6

2.2 Zaawansowana Redukcja Przestrzenna (Spatial Hashing z DSU) . . . 6

2.2.1 Zasada działania algorytmu . . . 6

2.2.2 Wyznaczanie centroidów i finalnych identyfikatorów . . . 6

2.2.3 Złożoność obliczeniowa i optymalizacje . . . 7

2.2.4 Implementacja . . . 8

2.3 Wyznaczanie Otoczki Wypukłej (Monotone Chain (algorytm Andrew)) . . . 8

2.3.1 Zasada działania algorytmu . . . 8

2.3.2 Złożoność obliczeniowa i optymalizacje . . . 8

2.3.3 Implementacja . . . 9

2.4 Wyznaczanie Otoczki Wypukłej (Graham's Scan) . . . 9

2.4.1 Zasada działania algorytmu . . . 9

2.4.2 Złożoność obliczeniowa i optymalizacje . . . 9

2.4.3 Implementacja . . . 10

2.5 Wyniki działania obu algorytmów . . . 10

3 Zadanie 2 **11**

3.1 Co wiemy z treści i regulaminu? . . . 11

3.1.1 Geometria toru i parametry przestrzenne (D 4.1.2, D 4.1.5) . . . 11

3.1.2 Układ współrzędnych i warunki brzegowe (D 4.1.6) . . . 11

3.1.3 Weryfikacja fizyczna w oparciu o pas jazdy (Test spójności) . . . 11

3.2 Analiza porównawcza i charakterystyka błędów pomiarowych sensorów . . . 12

3.3 Z czym pracujemy? . . . 12
3.3.1 Charakterystyka źródeł danych . . . 13
3.4 Analiza błędów . . . 14
3.4.1 Obserwacje ogólne . . . 14
3.4.2 Sensor 2 (rzadki): odbicie osi Y, outliery, szum . . . 14
3.4.3 Sensor 1 (gęsty): dryf . . . 14
3.5 Dlaczego takie przekształcenia? . . . 14

4 Cel i założenia **16**

5 Dane **16**
5.1 Wczytanie i podsumowanie . . . 16
5.2 Surowe trajektorie . . . 16

6 Analiza błędów czujników (podpunkt 2.1) **16**
6.1 Role czujników na podstawie próbkowania . . . 16
6.2 Orientacja osi Y . . . 16
6.3 Charakter błędu: residuum czujnik rzadki − gęsty . . . 17
6.3.1 Czujnik gęsty: dryf . . . 17
6.3.2 Czujnik rzadki: outliery i szum . . . 17
6.4 Podejrzewane źródła nieprawidłowości . . . 17

7 Metoda (podpunkt 2.2) **18**
7.1 Model sygnału . . . 18
7.2 Algorytm (filtr komplementarny typu “error-state”) . . . 18
7.3 Implementacja . . . 18

8 Wyniki i walidacja **18**
8.1 Trajektoria po fuzji . . . 18
8.2 Przebiegi w czasie . . . 18
8.3 Detekcja outlierów (filtr Hampela) . . . 18
8.4 Geometria toru jako kontrola poprawności . . . 23
8.5 Warunki brzegowe: błąd na początku i końcu . . . 24
8.6 Wrażliwość na parametry filtru Hampela . . . 24
8.7 Walidacja krzyżowa (leave-one-out) . . . 25
8.8 Kontrola fizyczna: przyspieszenie boczne . . . 25

9 Ograniczenia **26**

10 Uruchomienie programu C++ **26**

1 Wstęp

1.1 Użyte narzędzia

- **CLion** (JetBrains) – Środowisko programistyczne (IDE) wykorzystane do implementacji algorytmów i struktury danych w języku C++.
- **RStudio** – Środowisko pracy z językiem R, służące do analizy danych pomiarowych oraz zarządzania dokumentacją projektu
- **C++ (Standard C++)** – Język programowania użyty do implementacji wydajnych algorytmów redukcji przestrzennej (Spatial Hash, Greedy) oraz metod wyznaczania otoczki wypukłej (Graham Scan).
- **CMake** – System automatyzacji budowania i zarządzania konfiguracją projektu C++.
- **R Markdown & LaTeX (MiKTeX)** – Narzędzia do składu tekstu i automatycznego generowania sprawozdania technicznego w formacie PDF.
- **Biblioteka ggplot2 (R)** – Narzędzie graficzne wykorzystane do tworzenia czytelnych wykresów diagnostycznych oraz wizualizacji porównawczych punktów.

2 Zadanie 1

2.1 Algorytm Zachłanny (Greedy Clustering)

2.1.1 Zasada działania algorytmu

Algorytm opiera się na iteracyjnym łączeniu punktów, których odległość euklidesowa mieści się w zadanym progu odległości *threshold* (*eps*).

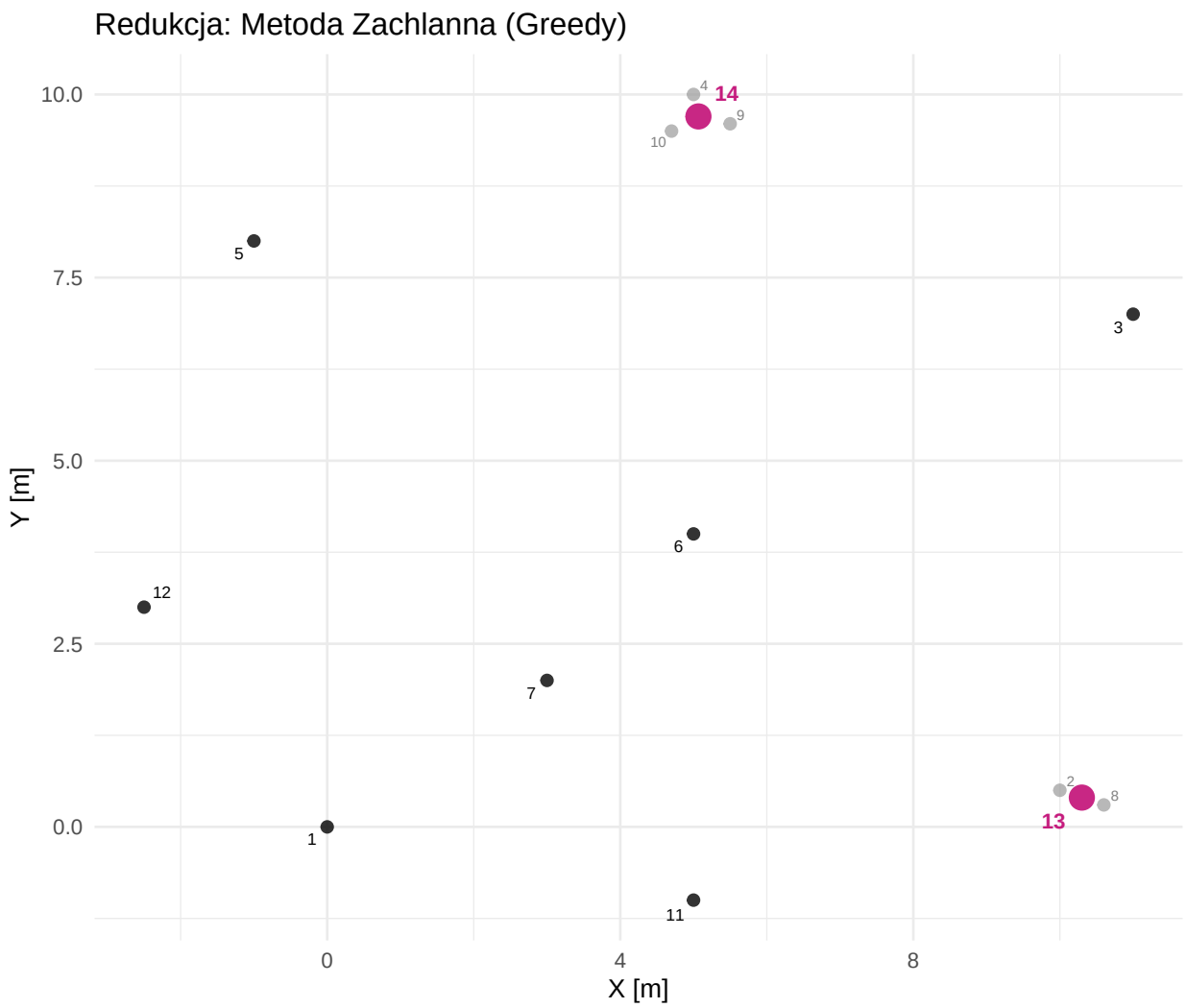
Proces przebiega w następujących krokach:

1. **Inicjalizacja:** Dla każdego punktu inicjalizowana jest lista jego oryginalnych identyfikatorów (`originalIds`), początkowo zawierająca wyłącznie jego własne `id`.
2. **Pętla iteracyjna fuzji (`while`):**
   - W każdej iteracji przeszukiwana jest para nierozłączonych jeszcze punktów.
   - Jeśli odległość między punktem bieżącym *current* a jego sąsiadem $currentPoints[j]$ spełnia warunek bliskości:
     $$\Delta x^2 + \Delta y^2 \leq threshold^2$$
     punkty te kwalifikują się do fuzji.
3. **Średnia ważona pozycji:** Nowa pozycja połączonego klastra wyznaczana jest jako średnia ważona, uwzględniająca liczbę pierwotnych punktów składających się na oba klastry:
   $$x_{new} = \frac{x_1 \cdot N_1 + x_2 \cdot N_2}{N_1 + N_2}$$
   Zapewnia to stabilność pozycji centroidu i zapobiega „pływaniu" wykrytych obiektów.
4. **Scalanie identyfikatorów:** Listy oryginalnych ID obu obiektów są łączone w jedną, a sąsiad oznaczany jest jako pochłonięty (`merged[j] = true`).
5. **Warunek stopu:** Pętla powtarza się dopóki w danej iteracji uda się dokonać jakiegokolwiek połączenia.

2.1.2 Złożoność obliczeniowa i optymalizacje

Pod względem teoretycznym klasyczny algorytm zachłanny charakteryzuje się wysoką złożonością czasową:

- **W najgorszym przypadku:** $\mathcal{O}(N^3)$ (wynikającą z wielokrotnego wykonywania przeszukiwań N^2 w pętli `while` aż do stabilizacji klastrów).
- **W przestrzeni pamięciowej:** $\mathcal{O}(N)$ na przechowywanie struktur wektorowych.


Redukcja: Metoda Zachlanna (Greedy)
10.0
7.5
5.0
2.5
0.0
Y [m]
0
4
8
X [m]
14
4
9
10
5
3
6
12
7
13
2
8
1
11

2.1.3 Implementacja

2.2 Zaawansowana Redukcja Przestrzenna (Spatial Hashing z DSU)

W celu przezwyciężenia ograniczeń wydajnościowych klasycznego algorytmu zachłannego, zaimplementowano wydajną metodę opartą na **siatce przestrzennej (*Spatial Hashing*)** oraz strukturze danych **Disjoint Set Union (DSU / Union-Find)**. Podejście to redukuje liczbę wymaganych porównań.

2.2.1 Zasada działania algorytmu

Algorytm dzieli przestrzeń 2D na dyskretną siatkę kwadratową, w której bok każdej komórki odpowiada progowi odległości ε (`eps`).

Proces przebiega w następujących krokach:

1. **Przypisanie do siatki:** Dla każdego punktu wyznaczane są współrzędne komórki (`cx`, `cy`) przy użyciu funkcji podłogi:
$$cx = \lfloor x/\varepsilon \rfloor, \quad cy = \lfloor y/\varepsilon \rfloor$$
Punkty umieszczane są w słowniku haszującym (`std::unordered_map`), co pozwala na natychmiastowy dostęp do obiektów w danej strefie.
2. **Przeszukiwanie sąsiedztwa i DSU:** Dla każdej niepustej komórki algorytm sprawdza wyłącznie **9 sąsiadujących komórek** (wliczając ją samą).
3. **Warunek bliskości:** Dla par punktów z komórki bieżącej oraz sąsiedniej testowany jest warunek odległości z użyciem kwadratów (bez operacji pierwiastkowania):
$$\Delta x^2 + \Delta y^2 \leq \varepsilon^2$$
Jeśli warunek jest spełniony, wywoływana jest operacja `unite` struktury DSU, łącząca punkty w spójny klaster.
4. **Kompresja ścieżki:** Struktura DSU zarządza relacjami w czasie niemal stałym dzięki mechanizmowi kompresji ścieżki (`path compression`) w operacji `find`.
5. **Grupowanie klastrów:** Po przetworzeniu siatki punkty są ostatecznie grupowane na podstawie wspólnego korzenia w DSU.

2.2.2 Wyznaczanie centroidów i finalnych identyfikatorów

Po zakończeniu fazy fuzji dla każdego wyznaczonego klastra wykonywane są operacje końcowe:

- **Centroid:** Pozycja nowego punktu wyliczana jest jako średnia arytmetyczna współrzędnych x i y wszystkich elementów należących do klastra.
- **Zarządzanie identyfikatorami:** Oryginalne ID punktów składowych są zbierane i sortowane rosnąco.
- Pojedyncze pachołki zachowują swoje pierwotne `id`, natomiast klastry powstałe z fuzji otrzymują nowe, sekwencyjne identyfikatory zaczynające się od wartości $maxOriginalId + 1$.

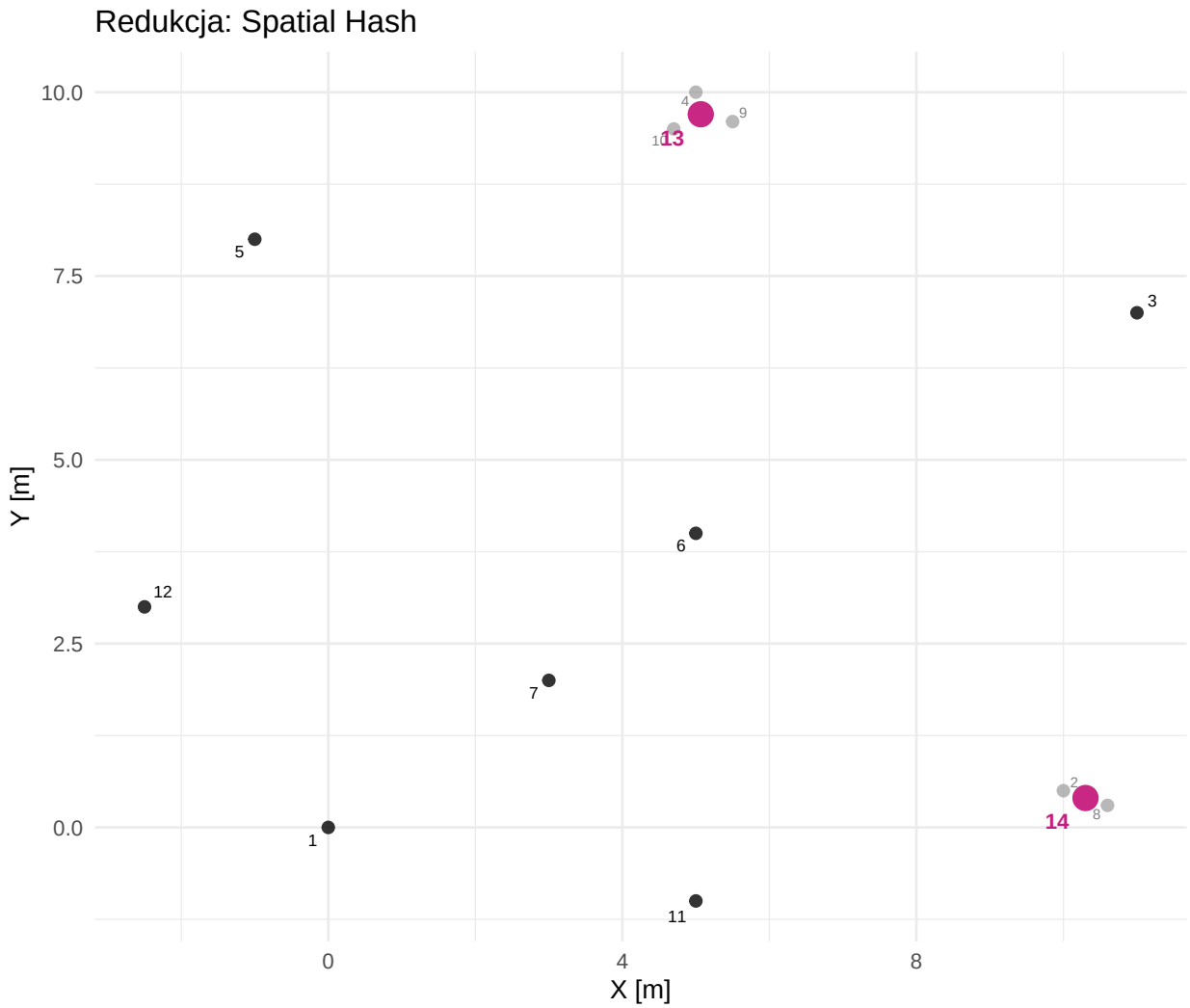
2.2.3 Złożoność obliczeniowa i optymalizacje

Pod względem teoretycznym podejście to zapewnia drastyczną poprawę wydajności:

- **W najgorszym przypadku:** $\mathcal{O}(N \cdot M)$ (gdzie M to średnia liczba punktów w sąsiadujących komórkach, co w praktyce dla równomiernie rozłożonych pachołków daje złożoność bliską liniowej $\mathcal{O}(N)$).
- **W przestrzeni pamięciowej:** $\mathcal{O}(N)$ na potrzeby struktury siatki oraz DSU.

W kodzie C++ zastosowano dodatkowe optymalizacje niskopoziomowe:

- **Użycie wspólnej funkcji kwadratu odległości (`calculateDistanceSquared`):** Całkowite wyeliminowanie kosztownych operacji `std::sqrt` i `std::pow` podczas testów sąsiedztwa w pętlach siatki.
- **Wydajne kontenery STL:** Wykorzystanie `std::unordered_map` do szybkiego haszowania komórek przestrzennych.
- **Bezpieczne przekazywanie danych:** Szerokie stosowanie stałych referencji (`const auto&`) eliminujących zbędne kopiowanie obiektów w pamięci. ## Wyznaczanie Otoczki Wypukłej (Convex Hull)



2.2.4 Implementacja

2.3 Wyznaczanie Otoczki Wypukłej (Monotone Chain (algorytm Andrew))

2.3.1 Zasada działania algorytmu

Otoczka wypukła wyznacza najmniejszy wielokąt wypukły, który zawiera wszystkie zredukowane punkty (centroidy pachołków) w przestrzeni dwuwymiarowej.

Proces przebiega w następujących etapach:

1. **Sortowanie punktów:** Zbiór wszystkich zredukowanych pachołków jest sortowany najpierw niemalejąco według współrzędnej x, a w przypadku jej równości – według współrzędnej y.
2. **Budowa dolnej i górnej otoczki:** Algorytm iteracyjnie przetwarza posortowane punkty, budując najpierw otoczkę dolną, a następnie górną (zgodnie z metodą łańcucha monotonnego / *Monotone Chain*).
3. **Test orientacji (iloczyn wektorowy):** W trakcie budowy konturu sprawdzany jest kierunek skrętu dla kolejnych trzech punktów za pomocą iloczynu wektorowego:

   $$(p_2.x - p_1.x) \cdot (p_3.y - p_1.y) - (p_2.y - p_1.y) \cdot (p_3.x - p_1.x)$$

   Jeśli orientacja nie spełnia warunku wypukłości (występuje skręt w „złą stronę"), poprzedni punkt jest usuwany z budowanej otoczki.
4. **Zamknięcie konturu:** Połączenie obu ścieżek tworzy spójną, uporządkowaną sekwencję wierzchołków definiujących zewnętrzną granicę rozmieszczenia pachołków.

2.3.2 Złożoność obliczeniowa i optymalizacje

Pod względem teoretycznym proces wyznaczania otoczki charakteryzuje się bardzo korzystną efektywnością:

- **W czasie:** $\mathcal{O}(N \log N)$, gdzie N to liczba zredukowanych punktów (główny narzut wynika z początkowego sortowania współrzędnych).
- **W przestrzeni pamięciowej:** $\mathcal{O}(N)$ na przechowywanie struktur wierzchołków tworzących wielokąt.

2.3.3 Implementacja

2.4 Wyznaczanie Otoczki Wypukłej (Graham's Scan)

2.4.1 Zasada działania algorytmu

Otoczka wypukła wyznacza najmniejszy wielokąt wypukły, który zawiera wszystkie zredukowane punkty (centroidy pachołków) w przestrzeni dwuwymiarowej.

Proces przebiega w następujących etapach:

1. **Wybór punktu bazowego (kotwicy):** Algorytm znajduje punkt wysunięty najdalej w dół (o najmniejszej współrzędnej y, a przy remisie najbardziej w lewo), który gwarantuje przynależność do otoczki, i ustanawia go punktem odniesienia.
2. **Sortowanie kątowe (biegunowe):** Pozostałe punkty są sortowane rosnąco według kąta biegunowego względem kotwicy (przeciwnie do wskazówek zegara), z wykorzystaniem iloczynu wektorowego oraz kwadratów odległości w przypadku współliniowości.
3. **Filtrowanie punktów współliniowych:** Z posortowanej listy eliminowane są punkty pośrednie leżące na tym samym promieniu od kotwicy, pozostawiając jedynie punkt najbardziej oddalony.
4. **Skan Grahama i test orientacji (stos):** Algorytm wykorzystuje strukturę stosu, do której kolejno trafiają przefiltrowane punkty. Kierunek skrętu dla kolejnych trzech punktów weryfikowany jest za pomocą iloczynu wektorowego:

   $$(p_2.x - p_1.x) \cdot (p_3.y - p_1.y) - (p_2.y - p_1.y) \cdot (p_3.x - p_1.x)$$

   Jeśli orientacja nie spełnia warunku wypukłości (występuje skręt w „złą stronę" lub linia prosta), punkt ze szczytu stosu jest usuwany, aż do zachowania poprawnej struktury wielokąta.

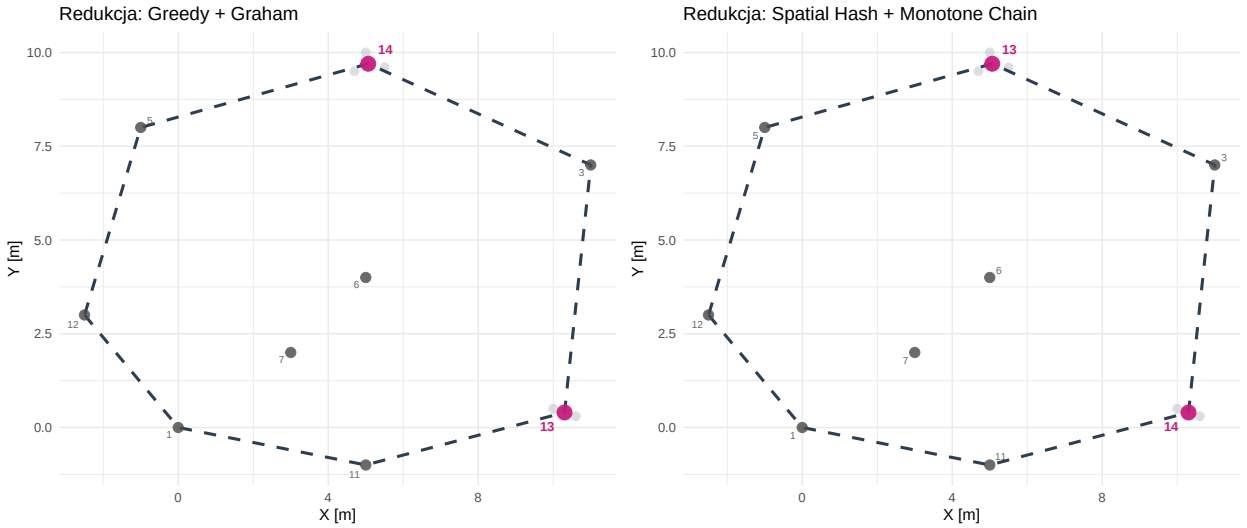
2.4.2 Złożoność obliczeniowa i optymalizacje

Pod względem teoretycznym proces wyznaczania otoczki charakteryzuje się bardzo korzystną efektywnością:

- **W czasie:** $\mathcal{O}(N \log N)$, gdzie N to liczba zredukowanych punktów (główny narzut wynika z początkowego sortowania kątowego punktów względem kotwicy).
- **W przestrzeni pamięciowej:** $\mathcal{O}(N)$ na przechowywanie struktur wierzchołków, wektora przefiltrowanego oraz stosu budującego wielokąt.

2.4.3 Implementacja

2.5 Wyniki działania obu algorytmów



Wyniki wyjściowe uzyskane dla obu zaimplementowanych algorytmów wykazują pełną zgodność z danymi referencyjnymi zawartymi w materiałach źródłowych.

3 Zadanie 2

3.1 Co wiemy z treści i regulaminu?

3.1.1 Geometria toru i parametry przestrzenne (D 4.1.2, D 4.1.5)

- **Kształt toru:** Konkurencja odbywa się na torze w kształcie cyfry 8, składającym się z dwóch stycznych okrągów wyznaczonych przez pachołki.
- **Szerokość pasa:** Pas drogi ma stałą szerokość wynoszącą **3 m**, ograniczoną wewnętrznym okręgiem o średnicy 15.25 m (promień $r_{min} = 7.625$ m) oraz zewnętrznym okręgiem o średnicy 21.25 m (promień $r_{max} = 10.625$ m).
- **Oś geometryczna pasa:** Nominalny promień przejazdu środkiem pasa wynosi:

$$r = \frac{7.625 + 10.625}{2} = 9.125 \text{ m}$$

- **Współrzędne środków okręgów:** Przyjęto układ, w którym środki okręgów znajdują się w odległości 18.25 m od siebie, co umieszcza je w punktach $(0, \pm 9.125)$ w lokalnym układzie współrzędnych.

3.1.2 Układ współrzędnych i warunki brzegowe (D 4.1.6)

- **Linia startu/mety:** Zgodnie z punktem D 4.1.6 regulaminu, linia pomiaru czasu (start/meta) łączy środki obu okrągów, co oznacza, że współrzędna $x = 0$ leży bezpośrednio na tej linii, a okręgi stykają się w okolicach punktu $(0, 0)$.
- **Warunki początkowe i końcowe:** Informacja z treści zadania o rozpoczęciu ruchu 15 m przed linią pomiaru ($x(0) = -15.0$ m) oraz zatrzymaniu 10 m za nią ($x(33) = 10.0$ m) nie jest założeniem arbitralnym – wynika bezpośrednio z geometrii rozmieszczenia stanowiska startowego względem punktu centralnego toru.

3.1.3 Weryfikacja fizyczna w oparciu o pas jazdy (Test spójności)

- Ponieważ pojazd fizycznie porusza się wewnątrz 3-metrowego korytarza, **żaden poprawny odczyt nie może trwale opuścić wyznaczonego pasa ruchu**.
- Analiza wykresów surowych danych wykazuje, że czujniki dotknięte błędem dryfu lub inwersji przekraczają te granice (np. gęsty sensor osiąga wartości rzędu $x = -15.7$ m, wykraczając poza dopuszczalny margines). Procent punktów mieszczących się w nominalnym pasie jazdy stanowi zatem obiektywną miarę jakości każdego z sensorów przed i po fuzji.

3.2 Analiza porównawcza i charakterystyka błędów pomiarowych sensorów

3.3 Z czym pracujemy?

Analiza opiera się na dwóch plikach danych pomiarowych dostarczonych w formacie CSV: `sensor1.csv` oraz `sensor2.csv`. Każdy z plików zawiera rejestrację przejazdu w strukturze kolumnowej:

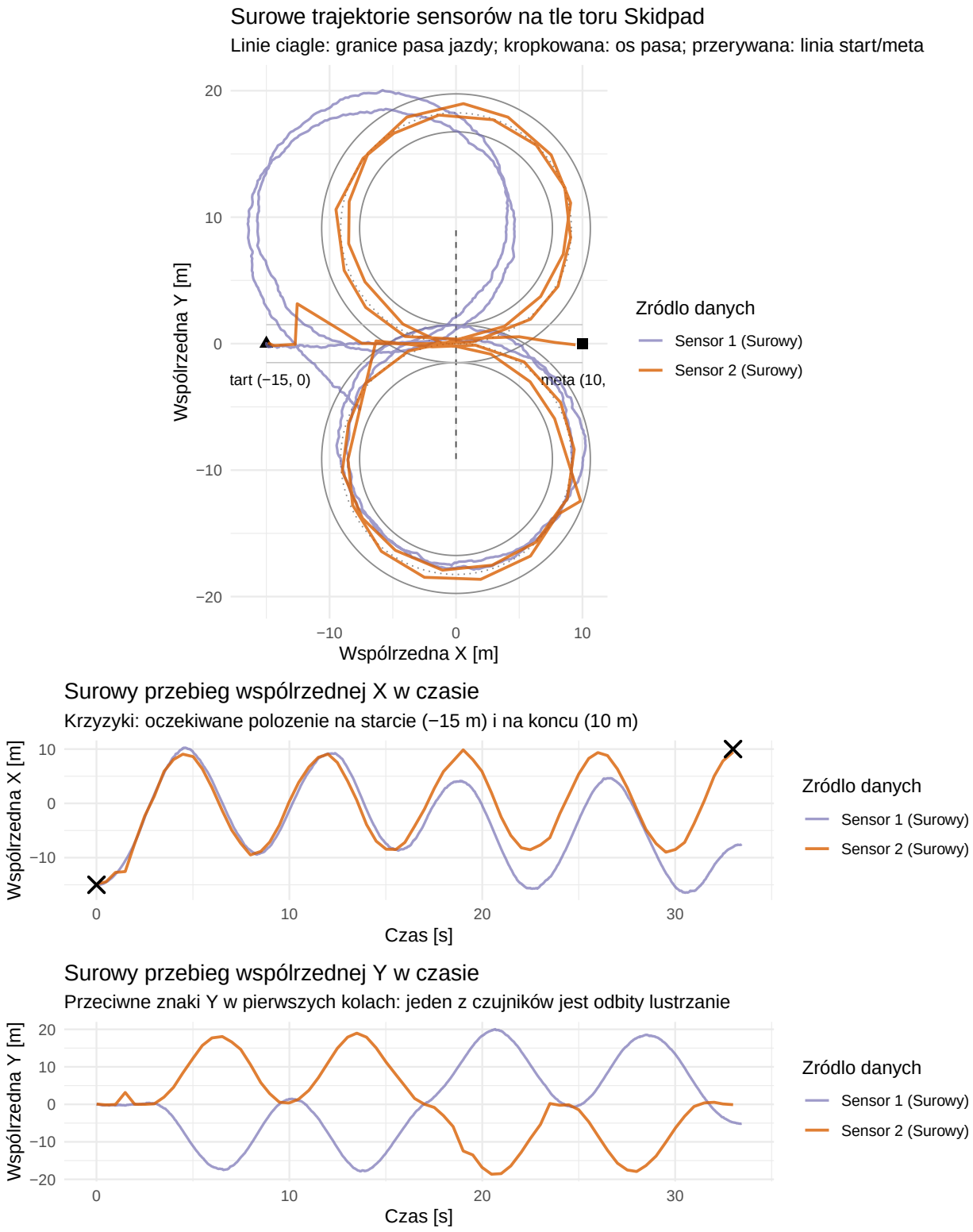
$$\text{timestamp}, x, y$$

Gdzie poszczególne pola oznaczają:

- `timestamp` – znacznik czasu w sekundach, określający moment dokonania pomiaru (dane obejmują przedział od ok. 0.008 s do 33.002 s).
- `x` – współrzędna wzdłużna położenia środka ciężkości bolidu w metrach (dodatnie wartości oznaczają ruch w przód, w tym punkty startowe i końcowe wynikające z regulaminu).
- `y` – współrzędna poprzeczna położenia pojazdu w metrach (dodatnie wartości oznaczają przemieszczenie w lewo).

3.3.1 Charakterystyka źródeł danych

Dostarczone sensory reprezentują dwie różne technologie pomiarowe/strumienie danych o odmiennej specyfice, co wymusza zastosowanie fuzji:



1. **Sensor 1 (Strumień wysokoczęstotliwościowy, ≈ 45 Hz):**
    - *Zalety:* Charakteryzuje się wysoką gęstością prókowania, dzięki czemu dobrze oddaje dynamikę jazdy
    - *Wady:* **inwersja osi Y** (współrzędne poprzeczne są odwrócone względem rzeczywistego kierunku jazdy, co tworzy obraz lustrzany) oraz skumulowany, nieliniowy **dryf** narastający w czasie.
2. **Sensor 2 (Strumień niskoczęstotliwościowy, ≈ 2 Hz):**
    - *Zalety:* Stanowi stabilną bazę odniesienia w makroskali. Jego punkty początkowe i końcowe idealnie korelują z warunkami brzegowymi toru Skidpad ($x(0) = -15$ m oraz $x(33) = 10$ m).
    - *Wady:* Niska częstotliwość próbkowania uniemożliwia bezpośrednie odtworzenie płynnego toru jazdy (między kolejnymi pomiarami występują duże przerwy czasowe rzędu 0.5 s), a dodatkowo strumień ten zawiera sporadyczne, skokowe zakłócenia (*outliery*).

3.4 Analiza błędów

3.4.1 Obserwacje ogólne

- Sensor 1 ma ok. 45 Hz, a Sensor 2 ok. 2 Hz
- Kanciaste koła Sensora 2 to cięciwy zamiast łuków, wynikają z rzadkiego próbkowania.

3.4.2 Sensor 2 (rzadki): odbicie osi Y, outliery, szum

- Odbicie osi Y. Na szczycie pierwszego koła Sensor 1 ma y $\approx -17{,}3$ m, a Sensor 2 y $\approx +18{,}1$ m.
- Outliery. Pojedyncze skoki o kilka metrów (np. przy t $\approx 1{,}49$ s), widoczne na Rys. 2 jako punkty odstające od gładkiej krzywej.
- Szum. Rozrzut rzędu dziesiątych części metra wokół trendu.
- Brak dryfu. Koniec toru przy ok. (9,45; $-0{,}09$), czyli ok. 0,6 m od (10, 0).

3.4.3 Sensor 1 (gęsty): dryf

- Obserwacja: zaczyna dokładnie w $(-15, 0)$, a przy t = 33 s jest w okolicy $(-8, -5)$, czyli ok. 18–19 m od (10, 0). Residuum na Rys. 2 rośnie gładko i bez skoków. Środki kół wędrują (Tabela 4). Pod koniec wychodzi poza pas jazdy (Tabela 4b).
- Wniosek: błąd zaczyna się od zera i narasta, więc to dryf, a nie stały offset.
- Hipoteza: całkowanie IMU lub żyroskopu bez korekty. - UWAGA TEN FRAGMENT ZOSTAŁ ZAPROPONOWANY PRZEZ MODEL GEMINI, NIE MAM OBECNIE NA TYLE WIEDZY DOMENOWEJ ŻEBY JEDNOZNACZNIE STWIERDZIĆ PRAWDZIWOŚĆ TEGO STWIERDZENIA

3.5 Dlaczego takie przekształcenia?

1. **Inwersja osi Y (Efekt lustrzanego odbicia):**
    - Na wykresie surowej trajektorii 2D oraz wykresie współrzędnej $Y w czasie widać, że jeden z sensorów rejestruje ruch w przeciwnym kierunku poprzecznym niż drugi (np. gdy jeden wskazuje wartości dodatnie, drugi ujemne o tej samej wielkości).
    - Przykładowo, w okolicach środka przejazdu różnica na osi Y między sensorami wynosiłaby kilkadziesiąt metrów zamiast ułamków metrów. Odbicie lustrzane jest błędem układu współrzędnych lub konwencji zapisu danych przez dany sensor.

- **Wniosek:** Żaden filtr matematyczny nie skoryguje odwróconego znaku. Wymaga to jawnej, deterministycznej korekty geometrycznej (inwersji osi Y jednego z sensorów) jako absolutnie pierwszego kroku przetwarzania.

2. **Rozbieżność dynamiki i dryf:**
   - Surowe wykresy w czasie ($X(t)$ oraz $Y(t)$) pokazują, że o ile oba czujniki początkowo startują z tego samego punktu ($x \approx -15$ m), o tyle wraz z upływem czasu Sensor 1 zaczyna nieliniowo "odjeżdżać" (dryfować) względem stabilnego Sensora 2.
   - Dodatkowo w strumieniu rzadszym pojawiają się pojedyncze, gwałtowne skoki (*outliery*).

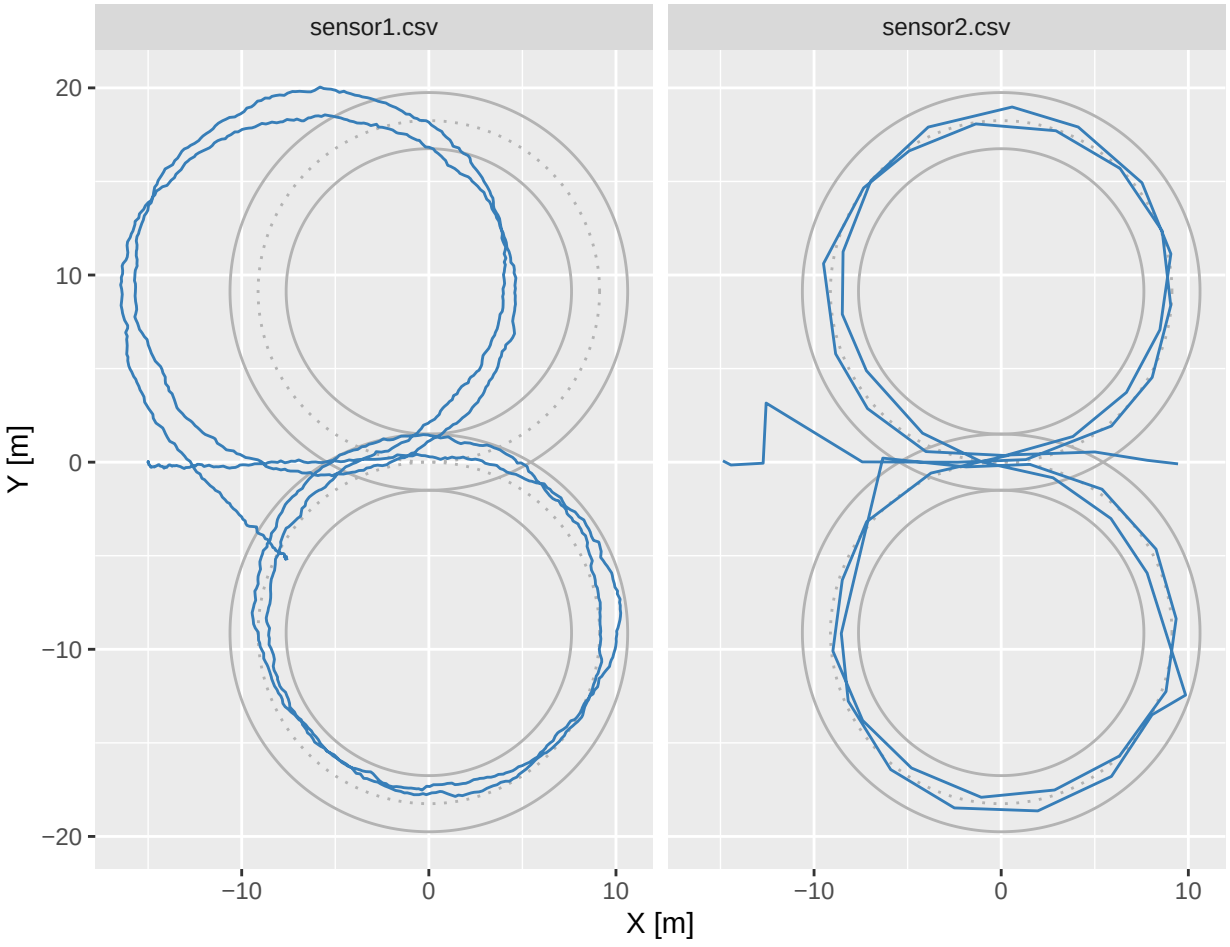


Figure 1: Rys. 1. Surowe trajektorie obu czujników (przed jakąkolwiek korektą). Linie ciągłe: granice pasa jazdy (r = 7,625 i 10,625 m), kropkowana: oś pasa (r = 9,125 m).

4 Cel i założenia

Celem jest wyznaczenie położenia środka ciężkości bolidu w **dowolnej chwili** $T \in [0, 33]$ s na podstawie dwóch błędnie działających czujników położenia (2D, widok z góry).

Założenia wynikające z treści zadania i regulaminu:

- dodatnie x to ruch w przód, dodatnie y to ruch w lewo,
- linia startu/mety to linia łącząca środki kół (regulamin D 4.1.6), więc przyjmuję $x = 0$ na tej linii; bolid startuje 15 m przed nią i zatrzymuje się 10 m za nią, czyli oczekiwane punkty graniczne to $(-15, 0)$ dla $t = 0$ i $(10, 0)$ dla $t = 33$,
- geometria toru (D 4.1.2, D 4.1.5): środki kół w odległości 18,25 m, czyli w $(0, \pm 9{,}125)$; średnica wewnętrzna 15,25 m ($R_{in} = 7{,}625$ m), zewnętrzna 21,25 m ($R_{out} = 10{,}625$ m); tor jazdy to pas **3 m** między nimi, więc oś pasa ma $R_0 = 9{,}125$ m,
- okrążenie to przejazd jednego koła od linii startu/mety do niej samej (D 4.1.6),
- kolejność kół: najpierw **prawe**, potem lewe. *Uwaga: ten warunek pochodzi z procedury przejazdu Skidpad w regulaminie (wpisz numer punktu); sam fragment D 4.1 opisuje tylko układ toru.*

5 Dane

5.1 Wczytanie i podsumowanie

Table 1: Tabela 1. Podstawowe parametry plików wejściowych.

Plik	Próbki	t od [s]	t do [s]	Mediana kroku [s]	Częstotliwość [Hz]
sensor1.csv	1339	0.000	33.447	0.025	40
sensor2.csv	67	0.008	33.002	0.501	2

5.2 Surowe trajektorie

Obserwacje wstępne: jeden czujnik jest bardzo gęsty i gładki, drugi rzadki, kanciasty i z pojedynczymi skokami. Kolejność kół jest w nich przeciwna (jeden zaczyna od dolnego, drugi od górnego).

6 Analiza błędów czujników (podpunkt 2.1)

6.1 Role czujników na podstawie próbkowania

Czujnik **gęsty**: sensor1.csv (40 Hz). Czujnik **rzadki**: sensor2.csv (2 Hz). Ruch ma pasmo rzędu ułamka herca (okres jednego koła ok. 7 s), więc nawet rzadki czujnik spełnia kryterium Nyquista dla wolnych składowych. Błąd interpolacji liniowej łuku to jednak $\approx R(1-\cos(\omega h/2))$, czyli dla kroku rzadkiego czujnika rzędu dziesiątek centymetrów, a dla gęstego pomijalny. Dlatego **tor** biorę z czujnika gęstego, a rzadki posłuży do korekty.

6.2 Orientacja osi Y

Pierwsze koło jest **prawe**, więc w poprawnym układzie ($y > 0$ = lewo) na szczycie pierwszego koła $y < 0$. - pierwsze prawe to też informacja z Claude

Table 2: Tabela 2. Kierunek pierwszego zakrętu.

Czujnik	y na szczycie 1. koła [m]	Wniosek
sensor1.csv (gęsty)	-17.52	zgodny z regulaminem
sensor2.csv (rzadki)	18.07	odbity lustrzanie

Table 3: Tabela 3. Zgodność czujników dla wariantów odwrócenia osi Y.

Wariant	Mediana normy residuum [m]
bez zmian (lub oba odwrócone)	16.55
odwrócony y: czujnik rzadki	3.15
odwrócony y: czujnik gęsty	3.15

Tabela 3 pokazuje, że **względna** orientacja czujników jest niespójna i wystarczy odwrócić jeden z nich. Który, rozstrzyga regulamin (Tabela 2): odwracam y w czujniku, który najpierw skręca w lewo. Uzasadnienie: odbicie lustrzane jest błędem systematycznym (izometria o wyznaczniku -1), którego żadna filtracja nie usunie. Wynik wymaga więc osobnego kroku.

6.3 Charakter błędu: residuum czujnik rzadki − gęsty

6.3.1 Czujnik gęsty: dryf

Residuum rośnie gładko i monotonicznie w skali całego przejazdu, a lokalnie zmienia się wolno. To sygnatura **dryfu**: błąd kursu $\theta(t)$ "obraca" całą trajektorię, więc błąd $\approx (R(\theta) - I)\, p(t)$ jest gładki i rośnie z czasem.

Po odrzuceniu punktów odstających średni przyrost błędu czujnika gęstego to ok. **0.33 m/s** (regresja liniowa normy residuum po czasie), a na końcu przejazdu błąd wynosi 17.5 m.

6.3.2 Czujnik rzadki: outliery i szum

Na Rys. 2 widać 4 punkty wyraźnie odstające od lokalnego trendu (rząd kilku metrów), przy typowym szumie rzędu dziesiątych części metra. Czujnik rzadki nie wykazuje narastającego błędu. Jego wiarygodność sprawdzam niezależnie w sekcji 4.5 (geometria toru) i 4.7 (leave-one-out).

6.4 Podejrzewane źródła nieprawidłowości

To **hipotezy** wnioskowane z kształtu błędów, a nie dowody.

Błąd	Obserwacja	Prawdopodobne źródło
Czujnik gęsty: dryf	gładkie, narastające rozjeżdżanie się toru; bez skoków	nieskorygowane całkowanie IMU / żyroskopu (błąd biasu żyroskopu → błąd kursu rosnący z czasem), odometria kołowa bez korekty zewnętrznej
Czujnik rzadki: outliery	pojedyncze skoki o kilka metrów, niska częstotliwość	odbiór GNSS: odbicia wielodrogowe (multipath), chwilowa utrata/zmiana satelitów

Błąd	Obserwacja	Prawdopodobne źródło
Czujnik rzadki: szum	rozrzut rzędu 0,1–0,3 m	typowy szum pomiaru pozycji GNSS
Odbicie osi Y u jednego z czujników	lustrzana kolejność kół	niespójna konwencja układu współrzędnych (np. y w prawo zamiast w lewo) lub odwrócony montaż

7 Metoda (podpunkt 2.2)

7.1 Model sygnału

$$p_f(t) = p(t) + d(t) + n_f(t), \qquad p_s(t) = p(t) + n_s(t) + o(t)$$

gdzie p to prawdziwa pozycja, d to dryf czujnika gęstego (f), n to szumy, a o to rzadkie outliery czujnika rzadkiego (s).

7.2 Algorytm (filtr komplementarny typu "error-state")

1. **Korekta układu odniesienia**: odwrócenie y w czujniku, który najpierw skręca w lewo (sekcja 3.2).
2. **Residuum** $e_i = p_s(t_i) - p_f(t_i)$ w chwilach pomiarów czujnika rzadkiego. Gęsty jest interpolowany liniowo. Wtedy $e \approx -d + n_s - n_f + o$, czyli estymata błędu czujnika gęstego.
3. **Filtr Hampela**: punkt i odrzucam, gdy jego odległość od mediany residuów z okna 5 próbek przekracza próg $\max(1\,\text{m},\ k \cdot 1{,}4826 \cdot \text{med}(\text{odchyleń}))$, $k = 5$. Mediana ignoruje pojedyncze skoki i nie myli wolnego trendu z anomalią (dla ciągu monotonicznego mediana to element środkowy). Stałe są heurystyką: współczynnik 1,4826 odpowiada skali rozkładu normalnego 1D, a tu mierzymy normę 2D. Dlatego w sekcji 4.6 jest test wrażliwości.
4. **Punkty skrajne**: okno Hampela jest tam jednostronne, więc mediana jest obciążona trendem. Punkt pierwszy i ostatni oceniam względem warunków brzegowych z zadania, czyli akceptuję go, jeśli leży w odległości ≤ 1 m od $(-15, 0)$ lub $(10, 0)$. Tolerancja 1 m jest mniejsza niż połowa szerokości 3-metrowego pasa wjazdu/wyjazdu (D 4.1.5), więc bolid nie mógł stać dalej od osi.
5. **Poprawka** $e(t)$: interpolacja liniowa przyjętych residuów (poza zakresem wartość skrajna).
6. **Wynik**: $\hat{p}(T) = p_f(T) + e(T)$.

Czego świadomie **nie** robię: wygładzania wyniku ani $e(t)$ (na okręgu skraca promień i tłumi składową dryfu oscylującą z częstotliwością ruchu) oraz filtru Kalmana (brak modelu pojazdu i niepewności czujników, więc macierze kowariancji byłyby dobrane arbitralnie).

7.3 Implementacja

8 Wyniki i walidacja

8.1 Trajektoria po fuzji

8.2 Przebiegi w czasie

8.3 Detekcja outlierów (filtr Hampela)

Odrzucono 3 z 67 próbek (próg 1.56 m); bramka punktów skrajnych zmieniła decyzję dla 1 punktów.

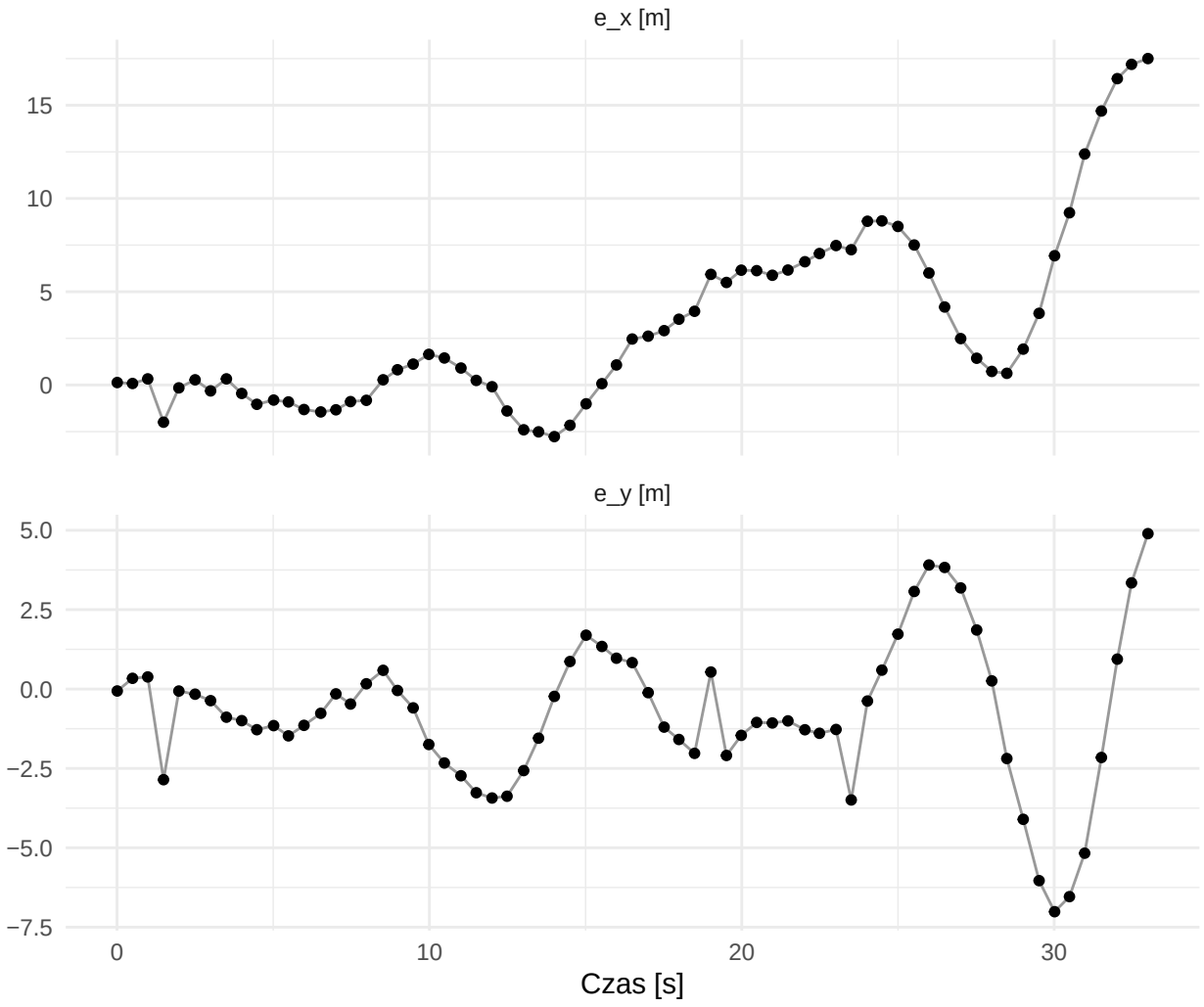


Figure 2: Rys. 2. Residuum e = rzadki − gęsty (po normalizacji osi Y). Gładka, narastająca krzywa to dryf; pojedyncze odstające punkty to outliery.

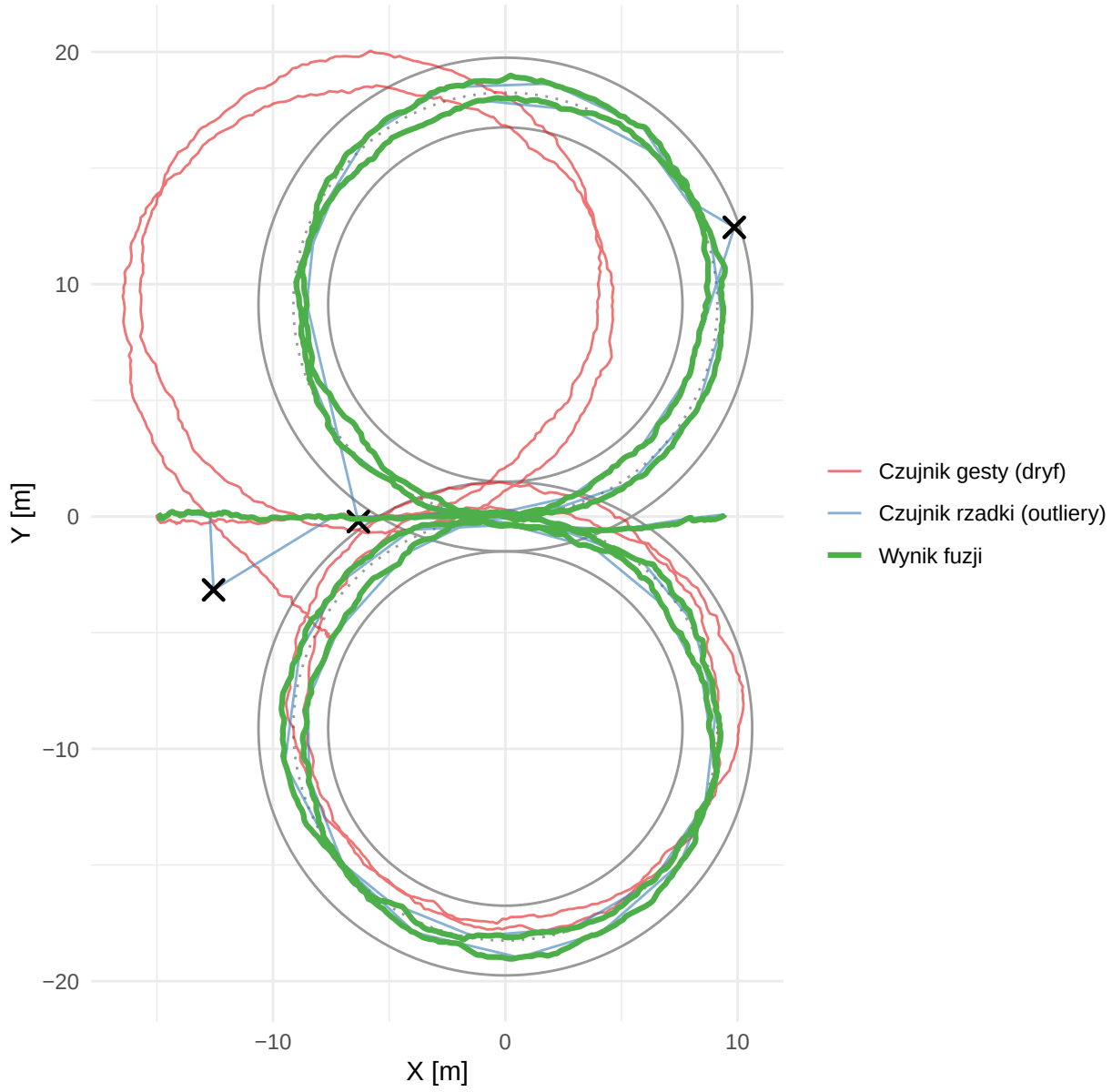


Figure 3: Rys. 3. Trajektoria po fuzji na tle czujników. Czarne krzyżyki: odrzucone punkty czujnika rzadkiego.

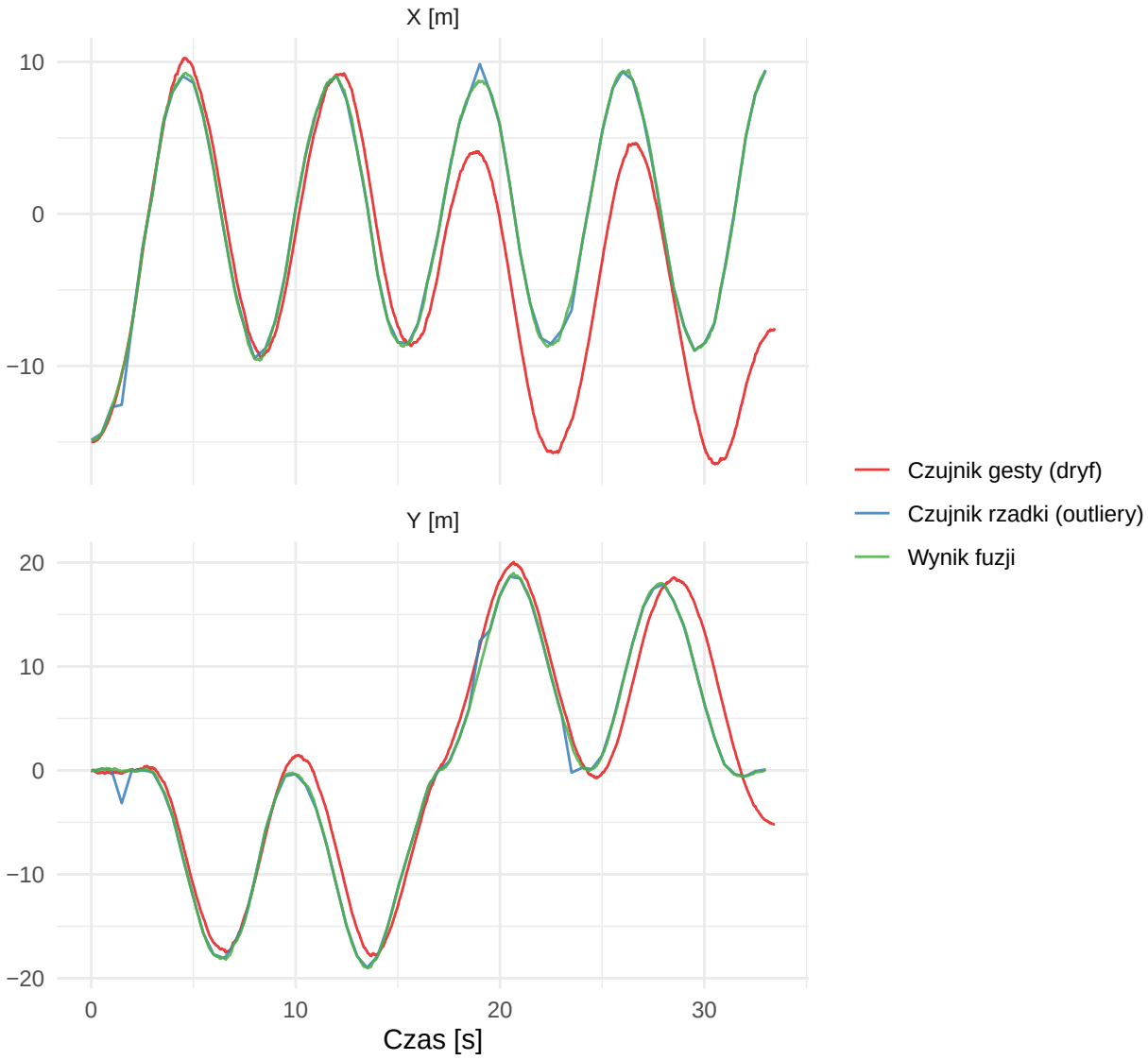


Figure 4: Rys. 4. Współrzędne X(t) i Y(t): czujniki i wynik fuzji.

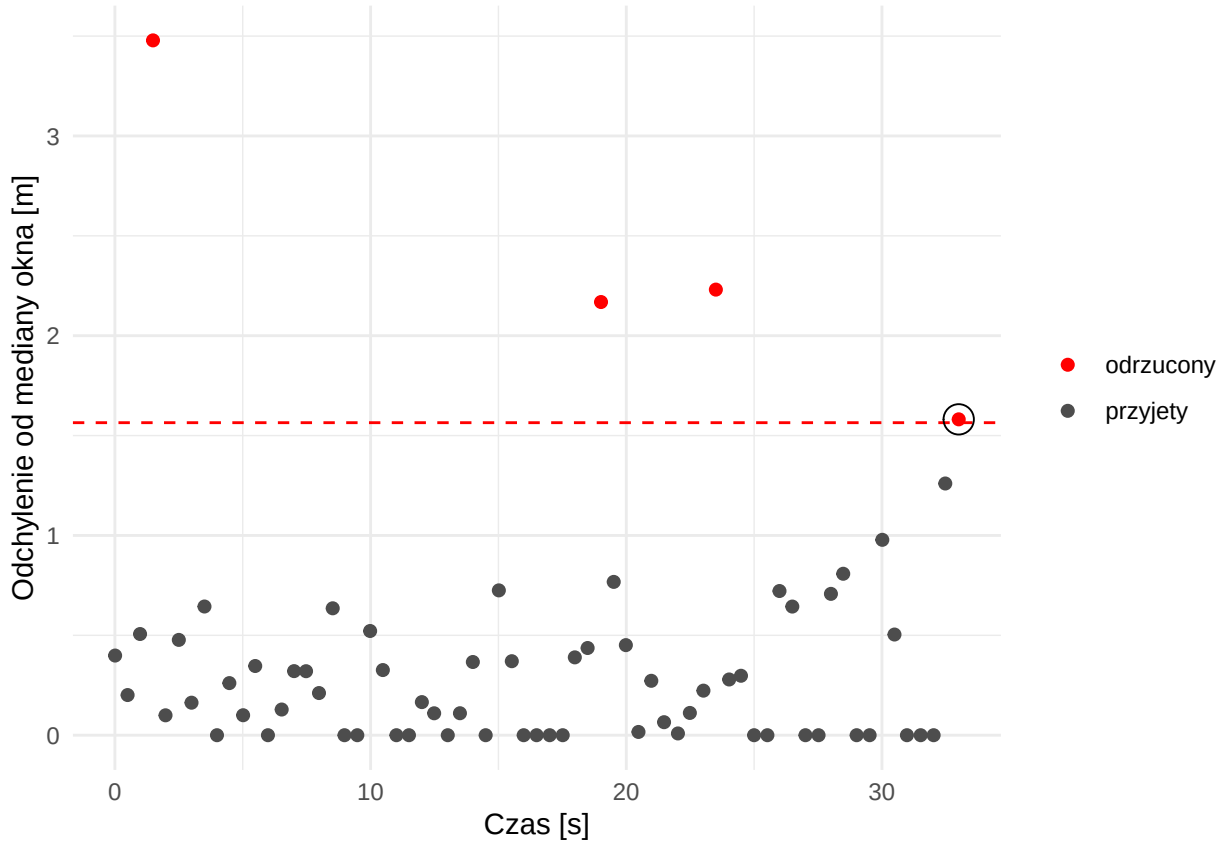


Figure 5: Rys. 5. Odchylenie residuum od mediany okna i próg odrzucenia. Okrąg: punkt skrajny, dla którego decyzję podjęto na podstawie warunków brzegowych.

8.4 Geometria toru jako kontrola poprawności

Do każdego okrążenia dopasowuję okrąg (metoda najmniejszych kwadratów Kåsy). Oczekiwane: $c_x \approx 0$, $c_y \approx \mp 9{,}125$, $r \approx 9{,}125$ m. Okna czasowe odczytano z przebiegu $X(t)$ (kolejne przejścia przez $x \approx 0$).

Table 5: Tabela 4. Dopasowane okręgi dla poszczególnych okrążeń.

Źródło	Okrążenie	n	cx	cy	r
gęsty	prawe 1	281	0.23	-8.05	9.42
gęsty	prawe 2	272	0.37	-8.38	9.19
gęsty	lewe 1	284	-5.83	9.70	10.05
gęsty	lewe 2	267	-6.06	8.32	10.37
rzadki	prawe 1	14	-0.13	-9.02	9.22
rzadki	prawe 2	14	0.18	-9.65	9.04
rzadki	lewe 1	12	0.10	9.57	9.04
rzadki	lewe 2	13	0.21	8.79	9.16
fuzja	prawe 1	701	-0.16	-9.03	9.25
fuzja	prawe 2	681	0.17	-9.63	9.04
fuzja	lewe 1	710	0.02	9.54	9.02
fuzja	lewe 2	671	0.31	8.72	9.18

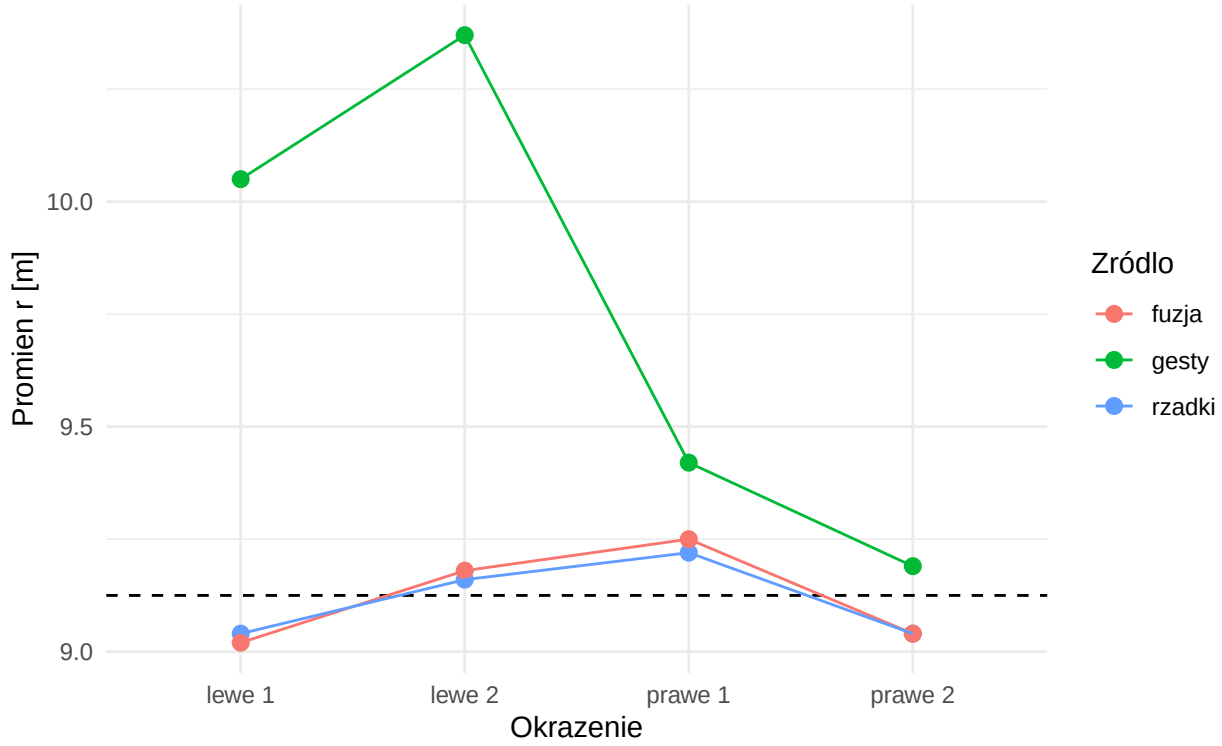


Figure 6: Rys. 6. Promień dopasowanego okręgu dla każdego okrążenia. Linia przerywana: 9,125 m.

Interpretacja: jeśli czujnik rzadki ma stabilne środki i promienie we wszystkich okrążeniach, a czujnik gęsty "wędruje", to potwierdza to założenie modelu (rzadki bez dryfu, gęsty z dryfem). Odchylenia rzędu dziesiątek centymetrów od 9,125 m nie muszą oznaczać błędu czujnika, bo bolid może jechać w dowolnym miejscu 3-metrowego pasa (D 4.1.5).

Twardy warunek z regulaminu: prawdziwy tor leży w pasie $R_{in} \le r \le R_{out}$ względem środka aktualnego koła. Dlatego mierzę, jaki procent punktów każdego źródła mieści się w pasie. Czujnik z dryfem musi go z czasem opuścić.

Table 6: Tabela 4b. Odsetek punktów w pasie jazdy (7.625 m $\le$ $r \le$ 10.625 m od środka koła).

Okrążenie	gęsty [%]	rzadki [%]	fuzja [%]
prawe 1	99	100	100
prawe 2	100	100	100
lewe 1	10	100	100
lewe 2	13	100	99

Okna `laps` leżą wewnątrz okrążeń (od przejścia przez linię start/meta do kolejnego), lekko przycięte, aby uniknąć przejść między kołami, gdzie tor jest styczny do osi X i nie opisuje go okrąg.

8.5 Warunki brzegowe: błąd na początku i końcu

Table 7: Tabela 5. Odległość od znanych punktów granicznych.

Wariant	Błąd względem (-15, 0) przy t = 0 [m]	Błąd względem (10, 0) przy t = 33 [m]
czujnik gęsty	0.00	18.68
czujnik rzadki	0.15	0.56
fuzja bez bramki punktów skrajnych	0.14	1.69
fuzja (finalna)	0.14	0.56

Błąd końcowy fuzji nie spada do zera, bo szum czujnika rzadkiego (rzędu 0,1–0,3 m) przechodzi do wyniku. Nie wymuszam zgodności z $(10, 0)$, aby nie ukryć rzeczywistego błędu czujnika.

8.6 Wrażliwość na parametry filtru Hampela

Table 8: Tabela 6. Liczba odrzuconych punktów i błąd końcowy dla różnych parametrów.

k	próg dolny [m]	check.names	Odrzucone	Błąd końca [m]
3	0.5	FALSE	5	0.56
4	0.5	FALSE	4	0.56
5	0.5	FALSE	3	0.56
6	0.5	FALSE	3	0.56
8	0.5	FALSE	1	0.56
3	1.0	FALSE	4	0.56
4	1.0	FALSE	4	0.56
5	1.0	FALSE	3	0.56
6	1.0	FALSE	3	0.56
8	1.0	FALSE	1	0.56
3	1.5	FALSE	3	0.56

k	próg dolny [m]	check.names	Odrzucone	Błąd końca [m]
4	1.5	FALSE	3	0.56
5	1.5	FALSE	3	0.56
6	1.5	FALSE	3	0.56
8	1.5	FALSE	1	0.56
3	2.0	FALSE	3	0.56
4	2.0	FALSE	3	0.56
5	2.0	FALSE	3	0.56
6	2.0	FALSE	3	0.56
8	2.0	FALSE	1	0.56

Jeśli liczba odrzuconych punktów i błąd końcowy są stabilne w szerokim zakresie parametrów, wybór $k = 5$, 1 m jest uzasadniony. Zmiany widoczne przy małych progach to fałszywe odrzucenia punktów w pobliżu ekstremów residuum.

8.7 Walidacja krzyżowa (leave-one-out)

Dla każdej przyjętej próbki czujnika rzadkiego (poza skrajnymi) buduję poprawkę **bez niej** i porównuję przewidywaną pozycję z pomiarem.

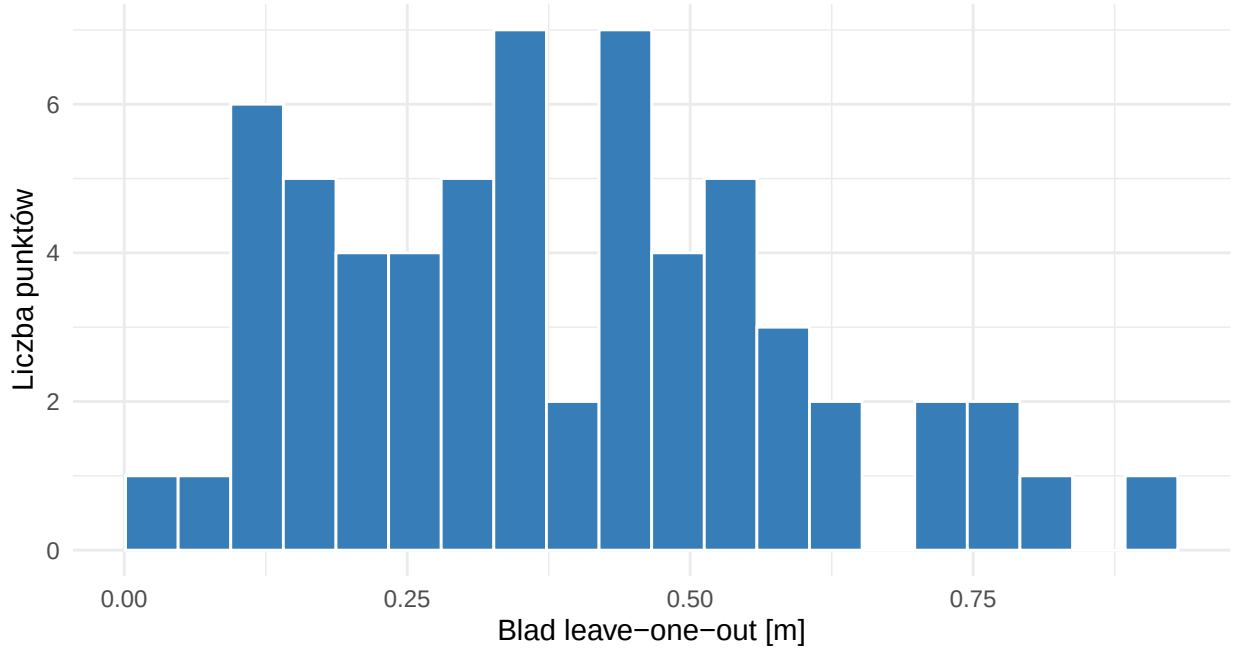


Figure 7: Rys. 7. Błąd leave-one-out: odległość między pomiarem rzadkim a pozycją przewidzianą bez tego pomiaru.

Mediana błędu: **0.37 m**, 95. percentyl: **0.76 m**. Jest to oszacowanie **z góry**, bo zawiera także szum samego pomiaru pominiętego punktu.

8.8 Kontrola fizyczna: przyspieszenie boczne

Mediana prędkości na prawych kołach: 7.9 m/s, więc przyspieszenie boczne $v^2/R_0 \approx$ **6.8 m/s²** (0.69 g). Wartość jest wiarygodna dla bolidu Formula Student, a wynik nie wymaga nierealistycznej dynamiki.

9 Ograniczenia

- Szum czujnika rzadkiego (rzędu 0,1–0,3 m) przechodzi do wyniku, bo $e(t)$ interpoluję przez surowe punkty. Wygładzanie zmniejszyłoby szum, ale zniekształciłoby dryf i skracało promień.
- Stałe filtru Hampela są heurystyczne (sekcja 4.6); okno 5 próbek może odrzucić poprawny punkt przy ekstremach residuum.
- Bramka punktów skrajnych zakłada, że linia pomiaru ma $x = 0$ i że czas przejazdu to 0–33 s.
- Źródła błędów (dryf IMU, multipath GNSS, konwencja osi) to hipotezy wnioskowane z sygnału, niezweryfikowane danymi z samych czujników.
- Wiarygodność czujnika rzadkiego opiera się na zgodności z geometrią toru i warunkami brzegowymi. Dla punktów pośrednich jest to argument, a nie dowód.

10 Uruchomienie programu C++

```
g++ -std=c++17 -O2 -Wall -Wextra skidpad_fusion.cpp -o build/skidpad
./build/skidpad data/DaneDoZadan/zad2/sensor1.csv data/DaneDoZadan/zad2/sensor2.csv
```

Program implementuje ten sam algorytm co powyższy kod R (kroki 1–6 z sekcji 4.2) i w pętli odpowiada na zapytania o T. Wartość kontrolna do porównania obu implementacji: `pozycja(26.821)` w R powinno dać to samo co wpisanie `26.821` w programie.

```
kable(round(pozycja(c(0, 6, 26.821, 33)), 3), row.names = FALSE,
      caption = "Tabela 7. Pozycje z implementacji w R dla kilku wartości T (do porównania z programem C
```

Table 9: Tabela 7. Pozycje z implementacji w R dla kilku wartości T (do porównania z programem C++).

t	x	y
0.000	-14.869	-0.061
6.000	2.814	-17.731
26.821	7.445	14.469
33.000	9.445	0.089

11 Wnioski

1. Czujniki mają **różne i uzupełniające się** błędy: gęsty dryfuje (0.33 m/s), a rzadki ma outliery i szum.
2. Jeden z czujników ma odwróconą oś Y. Wskazuje go regulamin (prawe koła najpierw), a nie sama niezgodność czujników.
3. Fuzja `gęsty + poprawka(rzadki − gęsty)` zachowuje dynamikę czujnika gęstego i poziom bezwzględny czujnika rzadkiego, bez przełączania i bez skoków.
4. Błąd leave-one-out ma medianę 0.37 m, a odległość końca od $(10, 0)$ wynosi 0.56 m.
5. Głównym ograniczeniem jest szum czujnika rzadkiego przechodzący do wyniku.